



算法设计与分析

第4章 · 基本的算法策略

4.1 迭代算法 · 4.2 蛮力法 · 4.3 分而治之算法

4.4 贪婪算法 · 4.5 动态规划 · 4.6 算法策略间的比较

基本思想：**用旧值算新值**，一般用于数值计算

常见迭代策略：累加、累乘

基本步骤：

- 1 确定迭代模型
- 2 建立迭代关系式
- 3 对迭代过程进行控制
 - 固定次数结束
 - 特定条件结束（精度达标）

兔子繁殖问题

一对兔子从出生后第三个月开始，每月生一对小兔子。假若兔子只生不死，一月份抱来一对刚出生的小兔子，问一年中每个月各有多少只兔子。

月份	1月	2月	3月	4月	5月	6月
数量	1	1	2	3	5	8

月份	7月	8月	9月	10月	11月	12月
数量	13	21	34	55	89	144

算法设计思路： $y_1 = y_2 = 1, y_n = y_{n-1} + y_{n-2}$

数据结构设计： 数组存储（保留全部） ↔ 变量滚动更新（仅前两月）

递推公式与循环不变式

$$y_n = y_{n-1} + y_{n-2}$$

变量说明：

a — 前 2 个月的数量

b — 前 1 个月的数量

c — 当前月份的数量

循环不变式： $c = a + b$

```
a = 1; b = 1;  
for (i = 3; i <= N; i++) {  
    c = a + b;  
    a = b;  
    b = c;  
}
```

兔子繁殖 — 完整代码

```
#include <stdio.h>

#define MONTHS 12

int main() {
    int a = 1, b = 1, c;
    printf("第1月: 1\n第2月: 1\n");
    for (int i = 3; i <= MONTHS; i++) {
        c = a + b;
        printf("第%d月: %d\n", i, c);
        a = b;
        b = c;
    }
    return 0;
}
```

运行结果：

第1月: 1 第2月: 1 第3月: 2 第4月: 3 第5月: 5 第6月: 8

第7月: 13 第8月: 21 第9月: 34 第10月: 55 第11月: 89 第12月: 144

例2：求两个整数的最大公约数

算法设计思路：

1) **短除法：** $\gcd(12, 18) = 2 \times 3 = 6$

2) **欧几里得算法（辗转相除法）：**

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

循环不变式：

$$c = a \bmod b, \quad a = b, \quad b = c$$

最大公约数 — 完整代码

```
int gcd(int a, int b) {  
    int c;  
    if (b == 0) return -1;  
    c = a % b;  
    while (c != 0) {  
        a = b;  
        b = c;  
        c = a % b;  
    }  
    return b;  
}  
  
int main() {  
    printf("gcd(18,12) = %d\n", gcd(18, 12));  
    printf("gcd(100,35) = %d\n", gcd(100, 35));  
    return 0;  
}
```

运行结果：

gcd(18,12) = 6

gcd(100,35) = 5

倒推法

迭代法：正推，由前向后解问题

倒推法：从后往前推解问题

由结果反推初始状态

例1：猴子吃桃问题

一只小猴子摘了若干桃子，每天吃现有桃的一半多一个，到第10天时就只有一个桃子了，求原有多少个桃？

倒推公式：

$$a_{n-1} = (a_n + 1) \times 2$$

```
int peach = 1;
for (int day = 10; day > 1; day--)
    peach = (peach + 1) * 2;
printf("原有 %d 个桃子\n", peach);
```

运行结果：原有 1534 个桃子

杨辉三角形（一维数组）

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

算法设计思路：

- 1) 一维数组 $A[1..i]$ 存储第 i 行
- 2) 只用一维数组，正推法将会覆盖上一行对应的值，不能求下一个值
反推法可以避免覆盖

递推公式：

$$A[i][j] = A[i-1][j-1] + A[i-1][j]$$

杨辉三角 — 正推失败 vs 反推成功

正推（覆盖上一行） ❌

$A[j] = A[j-1] + A[j]$

```
  1
 1 1
1 2 1
1 3 4 1    ← 第4行出错
```

正推累计误差 ❌

```
  1
 1 1
1 2 1
1 3 4 1
1 4 8 9 1  ← 错误越积越多
```

反推（从右向左） ✅

$A[j] = A[j-1] + A[j]$

```
  1
 1 1
1 2 1
1 3 3 1
```

```
A[1] = A[i] = 1;           // 首尾置1
for (int j = i - 1; j > 1; j--) // 从右向左倒推
    A[j] = A[j-1] + A[j];
```

穿越沙漠问题

用一辆吉普车穿越 1000 km 沙漠。总装油量 500 加仑，耗油 1 加仑/km。沙漠中无油库，必须用这辆车在沙漠中建立临时油库。

最省油方案：

① 每次从 a 点加满油出发

② a-b 之间来回 **奇数次**，最后一次朝 b 点走

③ a 点储油量 = a-b 之间耗油量 + b 点储油量

穿越沙漠 — 数学模型与分段倒推

变量说明:

k — 从 a 加满油向 b 出发的次数

$2k - 1$ — a-b 之间的来回次数

x — a-b 之间距离

S_1 — a 加油点的储油量

S_2 — b 加油点的储油量

数学模型:

$$S_1 = 500k$$

$$S_2 = 500k - (2k - 1)x$$

算法设计: 倒推法

1) 第一段

$$k = 1, S_2 = 0, x = 500, S_1 = 500$$

2) 第二段

$$k = 2, S_1 = 1000, x = 500/3$$

3) 第三段

$$k = 3, S_1 = 1500, x = 500/5$$

4) 直到 1000 km

穿越沙漠 — 完整代码

```
int main() {  
    int dis = 500, k = 1, oil = 500;  
    do {  
        printf("第%d个油库: 距%4dkm 储%4d\n",  
            k, 1000 - dis, oil);  
        k++;  
        dis += 500 / (2 * k - 1);  
        oil = 500 * k;  
    } while (dis < 1000);  
  
    oil = 500 * (k - 1) + (1000 - dis) * (2 * k - 1);  
    printf("起点: 距%4dkm 储%4d\n", 0, oil);  
    return 0;  
}
```

运行结果:

第1个油库: 距500km 储500

第2个油库: 距333km 储1000

第3个油库: 距200km 储1500

...

起点: 距 0km 储3836

迭代法解方程

基本思想：逐步逼近，从近似解到精确解

基本步骤：

确定初值 x_0

建立迭代关系 $f(x) = 0 \rightarrow x = \varphi(x)$

构造数列 $x_n = \varphi(x_{n-1})$

重复迭代，直到满足停止条件 $|x_n - x_{n-1}| < \varepsilon$

例1：迭代法求方程组根

算法说明： 方程组解的初值 $\mathbf{X} = (x_0, x_1, \dots, x_{n-1})$

迭代关系方程组为：

$$x_i = g_i(\mathbf{X}) \quad (i = 0, 1, \dots, n-1)$$

w 为解的精度

算法实现：

```
for (i = 0; i < n; i++)
    x[i] = 初始近似根;
do {
    k = k + 1;
    for (i = 0; i < n; i++)    y[i] = x[i];    // 保留旧值
    for (i = 0; i < n; i++)    x[i] = g[i]();    // 迭代计算
    for (c = i = 0; i < n; i++)
        c += fabs(y[i] - x[i]);    // 累计误差
} while (c > w && k < max_n);
for (i = 0; i < n; i++)
    printf("第%d个变量的近似根是: %f\n", i, x[i]);
```


例2：牛顿迭代法求根

牛顿迭代法解求形如 $ax^3 + bx^2 + cx + d = 0$ 方程的根

原理： 用切线近似代替曲线，逐步逼近根

$$f(x) = 0$$

$$f(x_0) + f'(x_0)(x - x_0) \approx 0$$

$$x = x_0 - \frac{f(x_0)}{f'(x_0)}$$

迭代公式：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

求解方程：

$$x^3 + 2x^2 + 3x + 4 = 0$$

牛顿迭代法 — 完整代码

```
#include <stdio.h>
#include <math.h>

float f(float a, float b, float c, float d) {
    float x1 = 1, x0, f0, f1;
    do {
        x0 = x1;
        f0 = ((a * x0 + b) * x0 + c) * x0 + d;
        f1 = (3 * a * x0 + 2 * b) * x0 + c;
        x1 = x0 - f0 / f1;
    } while (fabs(x1 - x0) >= 1e-6);
    return x1;
}

int main() {
    float root = f(1, 2, 3, 4);
    printf("方程的根为: %.6f\n", root);
    return 0;
}
```

运行结果： 方程的根为: -1.650629

例1：二分法求方程根

二分法求解 $\frac{x^3}{2} + 2x^2 - 8 = 0$ 在 $[0, 1]$ 上的近似根

前提条件： $f(x)$ 在求解区间 $[a, b]$ 上连续, 且 $f(a)$ 与 $f(b)$ 异号, 即 $f(a) \cdot f(b) < 0$

算法设计：

$$1) [a_0, b_0] = [a, b], c_0 = \frac{a_0 + b_0}{2}$$

若 $f(c_0) = 0$, 则 c_0 为根

若 $f(a_0) \cdot f(c_0) < 0$, 则 $[a_1, b_1] = [a_0, c_0]$

若 $f(b_0) \cdot f(c_0) < 0$, 则 $[a_1, b_1] = [c_0, b_0]$

2) 重复上述过程, 当 $f(c_n) = 0$ 时, 或区间 $[a_n, b_n]$ 足够小, 就认为找到了方程的根

二分法 — 完整代码

```
#include <stdio.h>
#include <math.h>

double f(double x) {
    return x * x * x / 2 + 2 * x * x - 8;
}

int main() {
    double x1 = 0, x2 = 2, x, f1, f2, fx;
    f1 = f(x1);    f2 = f(x2);

    if (f1 * f2 > 0) {
        printf("No root\n");
        return 0;
    }

    do {
        x = (x1 + x2) / 2;
        fx = f(x);
        if (fx == 0) break;
        if (f1 * fx > 0) { x1 = x; f1 = fx; }
        else             { x2 = x; f2 = fx; }
    } while (fabs(fx) >= 1e-4);

    printf("root = %.6f\n", x);
    return 0;
}
```